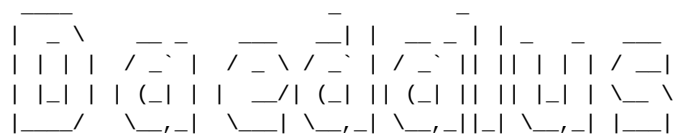


DAEDALUS

Server Module — Technical Reference

:: The labyrinth that builds itself ::



Spring Boot 3 • REST + WebSocket/STOMP • Plugin Runtime • Resilience4j

Module	daedalus-server
Artifact	daedalus-server-1.0.0-SNAPSHOT-exec.jar
Main Class	com.daedalus.server.DaedalusApp
Profiles	dev (default), prod, test
Port	8080 (configurable via SERVER_PORT)
Key Features	Circuit breaker, Redis (optional), STOMP, OpenAPI

1. Spring Boot Configuration Profiles

Daedalus uses Spring profiles to manage environment-specific behavior. The active profile is controlled by **DAEDALUS_PROFILE** environment variable (defaults to **dev**).

1.1 application-dev.yml (Development)

Key points: Redis is **disabled by default** (daedalus.redis.enabled=false). This prevents startup failures when no local Redis is running. Logging is verbose for com.daedalus package.

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
      timeout: 2s
  websocket:
    enabled: true

daedalus:
  plugins:
    scan-on-startup: true
  redis:
    enabled: false    # ← Critical for local dev
```

1.2 application-prod.yml (Production)

```
daedalus:
  plugins:
    directory: /var/daedalus/plugins
  redis:
    enabled: ${DAEDALUS_REDIS_ENABLED:true} # Redis ON by default in prod
```

1.3 application-test.yml

Uses Redis on port 6370 to avoid collision with dev. Plugin scanning is disabled for faster test execution.

2. Audit Fixes Implemented (Locked by Tests)

The following fixes were verified and locked in by dedicated unit tests. These address issues discovered during code audit.

2.1 Fix #1 — Generator ID on Circuit-Breaker Fallback

Problem: When the Resilience4j circuit breaker opened and the fallback (BinaryTreeGenerator) was used, the REST response still reported the *requested* generatorId (e.g. "astar") instead of the *actual* one ("binary-tree").

Solution: MazeGenerationService.fallback() now returns a Cached object whose MazeMetadata contains the correct generatorId from the algorithm that actually executed. MazeController.generate() reads from metadata().generatorId() rather than echoing the request.

Tests: MazeGenerationServiceFallbackTest + MazeControllerGeneratorIdTest

2.2 Fix #3 — RedisConfig Conditional Activation

Problem: RedisConfig always contributed beans, causing Spring Boot to attempt Redis connection even in dev profile where daedalus.redis.enabled=false.

Solution: Added @ConditionalOnProperty(name = "daedalus.redis.enabled", havingValue = "true", matchIfMissing = false) to RedisConfig. When the property is absent or false, the entire @Configuration class is skipped.

Test: RedisConfigConditionalTest — verifies beans are absent when disabled, present when enabled.

2.3 Plugin Failure STOMP Bridge

Requirement: When any plugin fails during DISCOVER, INIT, REGISTER_ALGORITHMS, START, or STOP, the frontend must receive a real-time notification on /topic/plugins/failures so it can show toasts/banners.

Implementation: MazeWebSocketController listens for PluginFailedEvent via @EventListener, transforms it into PluginFailedFrame, and publishes via SimpMessagingTemplate.convertAndSend().

Test: MazeWebSocketControllerPluginFailedTest — verifies correct topic and frame content.

3. REST API Reference (v1)

All endpoints are under **/api/v1** (note the version prefix in controller tests). DTOs are defined in `com.daedalus.api.dto` and mirrored in the frontend TypeScript package.

Method	Path	Request	Response	Description
POST	/api/v1/maze/generate	GenerateRequest	GenerateResponse	Generate new maze (with circuit-breaker)
GET	/api/v1/maze/{id}	—	GenerateResponse	Retrieve cached maze
POST	/api/v1/maze/{id}/session	—	SessionResponse	Create playable session
POST	/api/v1/maze/{id}/solve/{solverId}	—	SolveResponse	Run solver algorithm
POST	/api/v1/session/{id}/move	MoveRequest	— (or WS update)	Player move in session
GET	/api/v1/plugins	—	List<PluginInfo>	List loaded plugins + health

3.1 Core DTOs (Java Records)

GenerateResponse — `id`, `generatorId` (actual, even on fallback), `rows`, `cols`, `seed`, `tiles: char[][]` **SolveResponse** — `solverId`, `path: List`, `visited`, `explored`, `elapsedMs`, `success` **PluginInfo** — `id`, `state` (`DISCOVERED`|`INITIALIZED`|`STARTED`|`STOPPED`|`FAILED`), `manifest`, `error` **SessionResponse** — `sessionId`, `mazeId`, `position: Point` **PluginFailedFrame** — `pluginId`, `pluginVersion`, `phase`, `errorClass`, `errorMessage`, `timestamp`

4. WebSocket / STOMP Topics

Destination	Frame Type	Purpose
/topic/plugins/failures	PluginFailedFrame	Real-time plugin crash notifications (toasts)
/topic/maze/{id}/state	GeneratedFrame	Maze generation complete (rows/cols/generatorId)
/topic/maze/{id}/solver	SolvedFrame	Solver finished run (pathLength, success)
/topic/session/{id}/player	MoveFrame	Player position updates (from/to Point)

All frames are published via `SimpMessagingTemplate`. Frontend should use `@stomp/stompjs` or `sockjs-client` + `Stomp`.

5. Key Dependencies (pom.xml highlights)

• `spring-boot-starter-web` + `websocket` + `security` + `validation` + `actuator` • `spring-boot-starter-data-redis` (Lettuce) — gated by `daedalus.redis.enabled` • `resilience4j-spring-boot3` — circuit breaker on maze generation • `caffeine` — maze cache (256 entries, 30 min TTL) • `springdoc-openapi-starter-webmvc-ui` — Swagger UI at `/swagger-ui.html` • `micrometer-registry-prometheus` — `/actuator/prometheus`

The executable fat JAR is published with classifier **exec** so the thin JAR remains usable as a compile dependency for `daedalus-desktop` and other modules.

This document was auto-generated from the provided DTOs, configuration files, and audit test cases. For the full frontend TypeScript DTOs, see `daedalus-api-dtos.ts` in the `artifacts` folder.